k-nearest neighbor (k-NN)

k-NEAREST NEIGHBOR (k-NN)

- Is a nearest neighbor learning method
- kNN is also called a **lazy learner** algorithm because it **does not learn from the training dataset immediately;** instead, it stores the dataset, and at the time of classification, it acts on the dataset
- Its also called instance based/ memory based learning method **since it needs memory to store the dataset** in the training period and it predicts the class label of a test instance only when a new instance is given for classification, thus delays the processing of the training dataset

k-NEAREST NEIGHBOR (k-NN)

- So here, during training, no learning happens and **during classification phase it does the processing of the training dataset and classifies the test instance**
- It is a **non-parametric** and **supervised learning classifier**
- It handles both **classification and regression**
- It is **commonly used for classification**

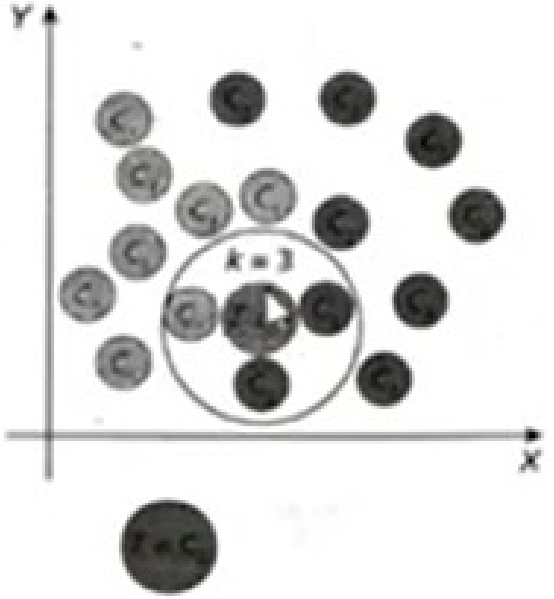
Working of k-NEAREST NEIGHBOR algorthm

The **principle behind k-NN algorithm is to predict the label of a new data point** (or test data point) by

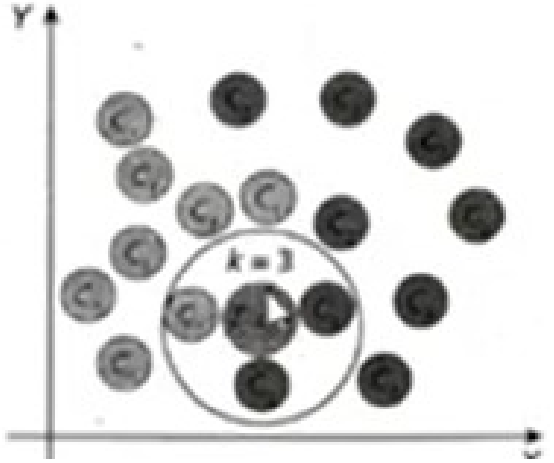
- considering the labels of the nearest neighbors

Here *k* stands for the number of nearest neighbors to be considered for making the prediction

- Let us say **all** the circles in the diagram are the **data points stored during** the training period
- Suppose that we have taken k=3 that means **out of n distances during testing, only 3 nearest neighbors**
- are considered for prediction



Working of k-NEAREST NEIGHBOR algorithm general steps



Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

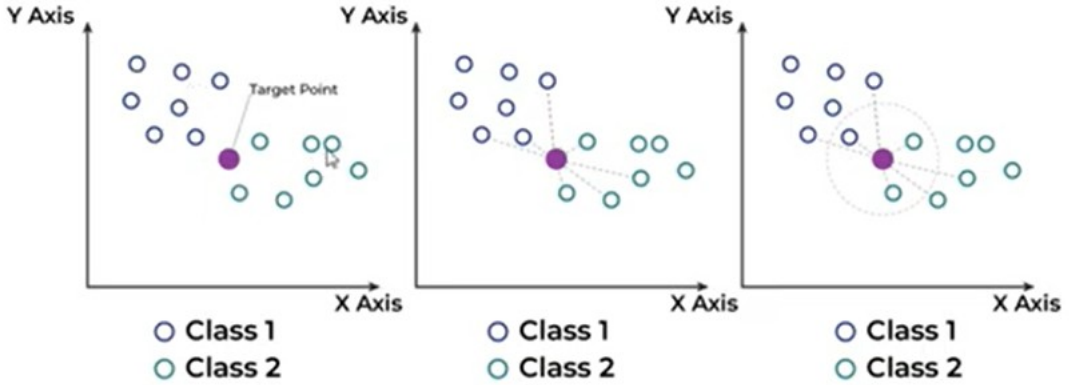
- To measure the similarity between target and training data points Euclidean distance is used.
- Distance is calculated between data points in the dataset and target point.
- Eg: Euclidean distance, Manhattan distance, Minkowski ditance

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

Working of k-NEAREST NEIGHBOR algorithm



k-NEAREST NEIGHBOR (k-NN)

Algorithm 4.1: *k*-NN

nputs: Training dataset T, distance metric d, Test instance t, the number of nearest neighbors k

Output: Predicted class or category

Prediction: For test instance t,

1. For each instance i in T, compute the distance between the test instance t and every other instance i in the training dataset using a distance metric (Euclidean distance).

   [Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as dist $((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]

   [Categorical attributes (Binary) - Hamming Distance: If the value of the two instances is same, the distance d will be equal to 0 otherwise $d = 1$.]

2. Sort the distances in an ascending order and select the first k nearest training data instances to the test instance.

3. Predict the class of the test instance by majority voting (if target attribute is discrete valued) or mean (if target attribute is continuous valued) of the k selected nearest instances.

k-NEAREST NEIGHBOR (k-NN) applications

- Applications of kNN are

- recommendation system in Amazon or Netflix
- Spam detection
- Customer segmentation
- Speech recognition

k-NEAREST NEIGHBOR (k-NN) advantages

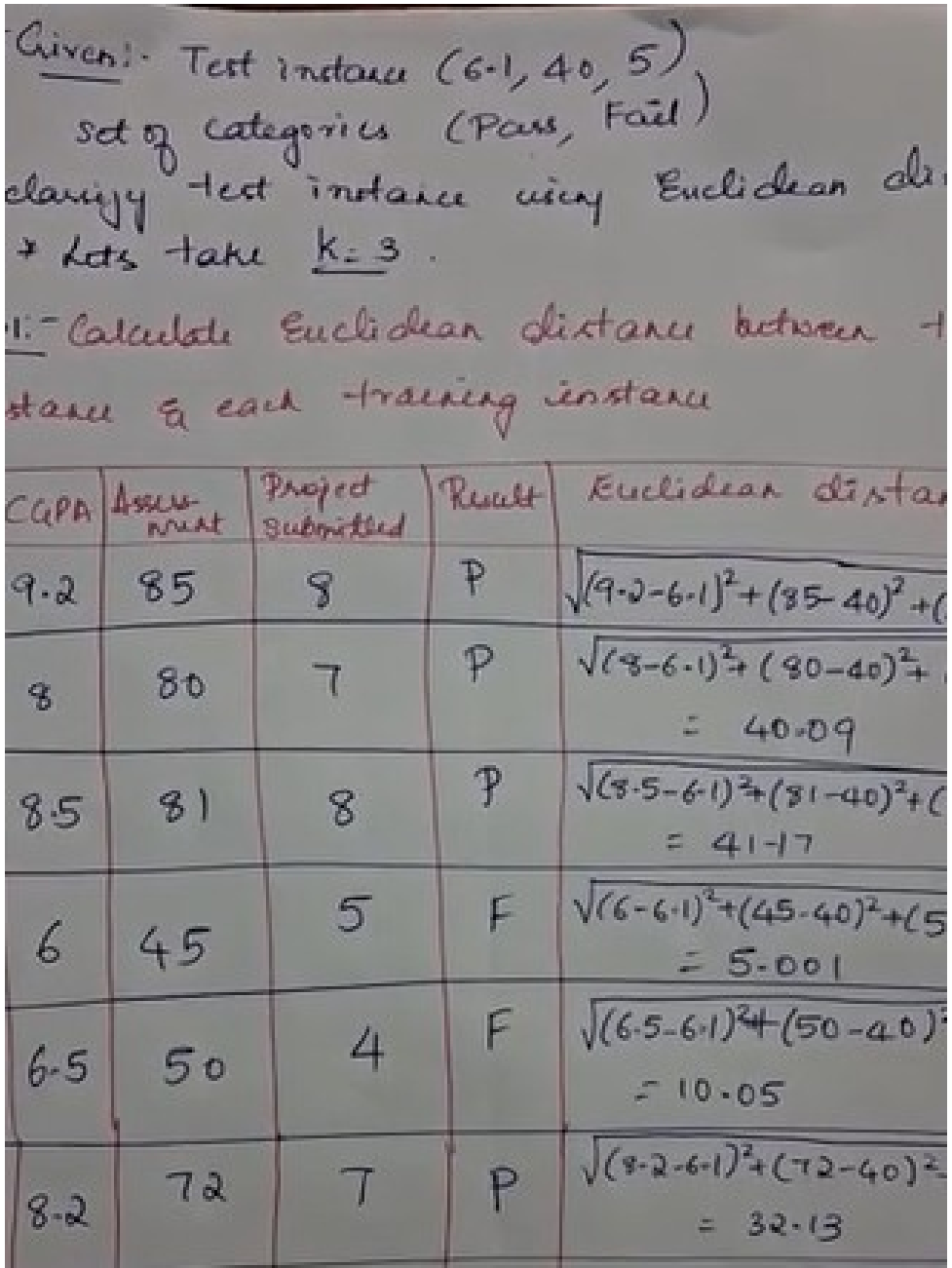
- **Advantages** of kNN are :

- Easy to implement
- No training required
- Few hyperparameters
- Flexible

Disadvantages of kNN are :

- Does not scale well to large datasets
- When no. of features increases, it struggles to classify
- Prone to overfitting

How to solve classification using k-NN?

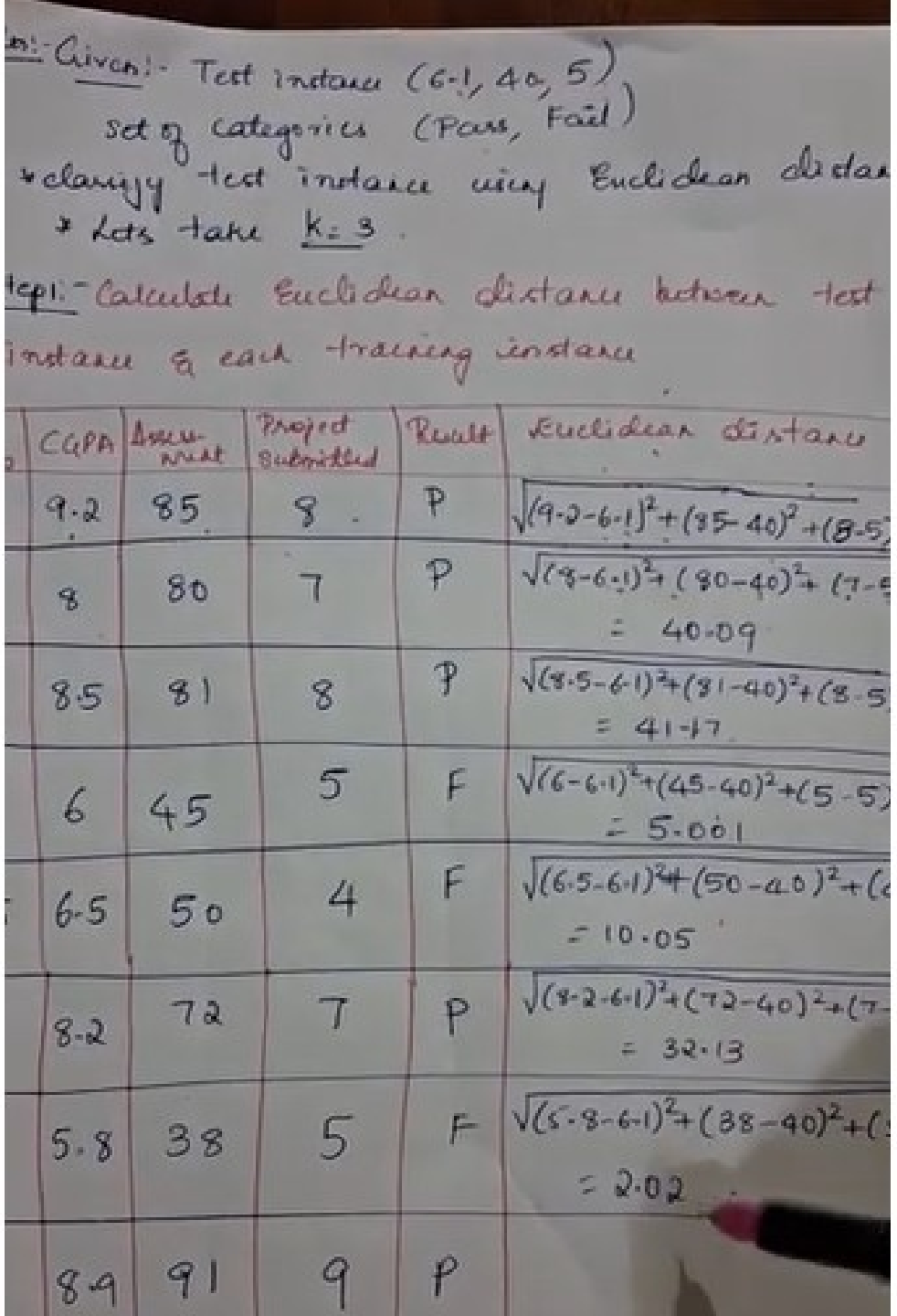


Given:- Test instance (6.1, 40, 5)
set of categories (Pass, Fail)
classify test instance using Euclidean di
* Lets take K=3.

1:- Calculate Euclidean distance between t
stance & each training instance

CGPA	Assignment	Project submitted	Result	Euclidean distan
9.2	85	8	P	$\sqrt{(9.2-6.1)^2+(85-40)^2+(}$
8	80	7	P	$\sqrt{(8-6.1)^2+(80-40)^2+}$ = 40.09
8.5	81	8	P	$\sqrt{(8.5-6.1)^2+(81-40)^2+(}$ = 41.17
6	45	5	F	$\sqrt{(6-6.1)^2+(45-40)^2+(5}$ = 5.001
6.5	50	4	F	$\sqrt{(6.5-6.1)^2+(50-40)^2}$ = 10.05
8.2	72	7	P	$\sqrt{(8.2-6.1)^2+(72-40)^2}$ = 32.13

How to solve classification using k-NN?

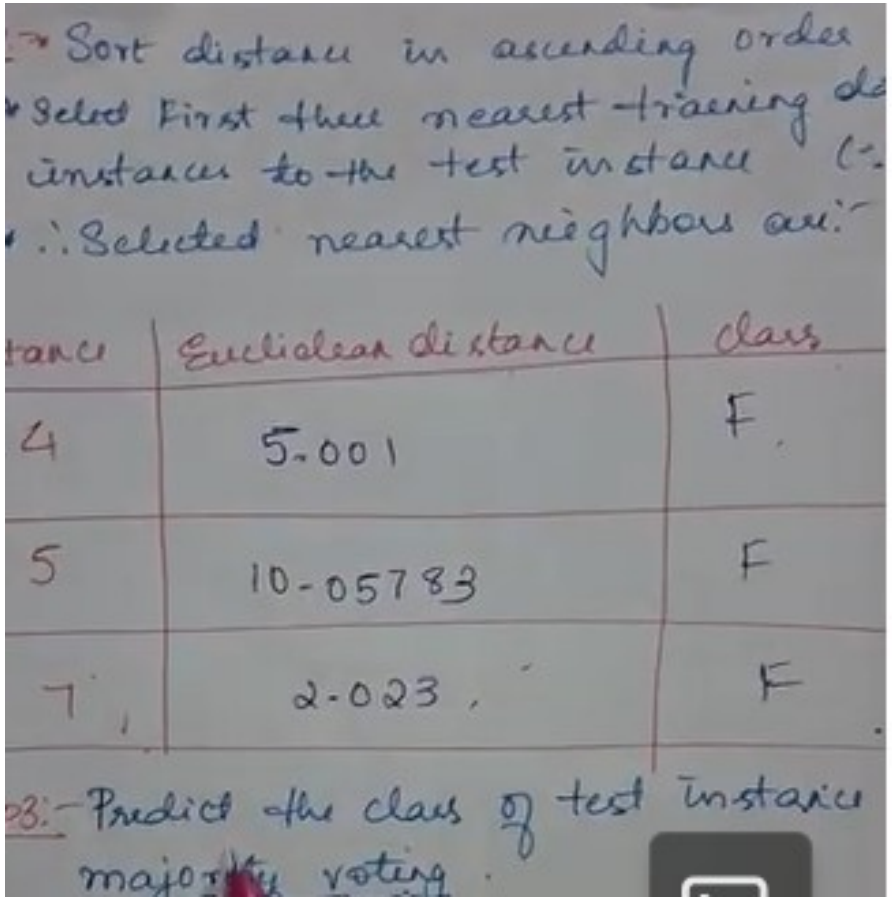


Given:- Test instance (6.1, 40, 5)
Set of categories (Pass, Fail)
* classify test instance using Euclidean dista
* Lets take k = 3.

Step1:- Calculate Euclidean distance between test instance & each training instance

CGPA	Assignment	Project Submitted	Result	Euclidean distance
9.2	85	8	P	$\sqrt{(9.2-6.1)^2+(85-40)^2+(8-5)}$
8	80	7	P	$\sqrt{(8-6.1)^2+(80-40)^2+(7-5}$ = 40.09
8.5	81	8	P	$\sqrt{(8.5-6.1)^2+(81-40)^2+(8-5}$ = 41.17
6	45	5	F	$\sqrt{(6-6.1)^2+(45-40)^2+(5-5)}$ = 5.001
6.5	50	4	F	$\sqrt{(6.5-6.1)^2+(50-40)^2+(}$ = 10.05
8.2	72	7	P	$\sqrt{(8.2-6.1)^2+(72-40)^2+(7-}$ = 32.13
5.8	38	5	F	$\sqrt{(5.8-6.1)^2+(38-40)^2+(}$ = 2.02
8.9	91	9	P	

k-NEAREST NEIGHBOR (k-NN) CLASSIFIER



→ Sort distance in ascending order
* Select First three nearest training da
instances to the test instance (
∴ Selected nearest neighbors are:-

tance	Euclidean distance	class
4	5.001	F
5	10-05783	F
7,	2-023.	F

3:- Predict the class of test instance
majority voting.

Q. From the given data-set find (x,y) = 57,170 whether belongs to Under or Normal Weights

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?

Solution:

In this approach we are going to use Euclidean distance formulae, n(no of records)=9 and assuming K value as 3

$d1 = \sqrt{((x2-x1)^2 + (y2-y1)^2)}$

x1=167,y1=51 and x2=170,y2=57

$d1 = \sqrt{((170-167)^2 + (57-51)^2)}$

$d1 = 6.7$

$d2 = \sqrt{((x2-x1)^2 + (y2-y1)^2)}$

x1=183,y1=56 and x2=170,y2=57

$d2 = \sqrt{((170-183)^2 + (57-56)^2)}$

$d2 = 13$

$d3 = \sqrt{((x2-x1)^2 + (y2-y1)^2)}$

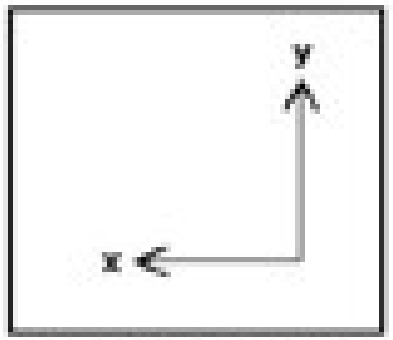
x1=176,y1=69 and x2=170,y2=57

$d3 = \sqrt{((170-176)^2 + (57-69)^2)}$

Distances

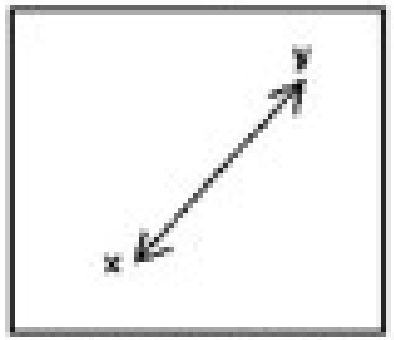
- Manhattan Distance

|X1-X2| + |Y1-Y2|



Manhattan

- Euclidean Distance
- $\sqrt{(x1-x2)^2} + \sqrt{(y1-y2)^2}$

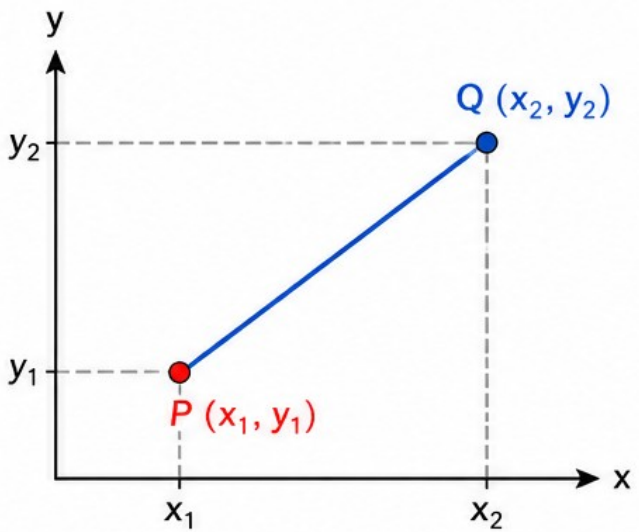


Euclidean

Distance Measures in 2D

1. Euclidean Distance

$$d_E(P,Q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

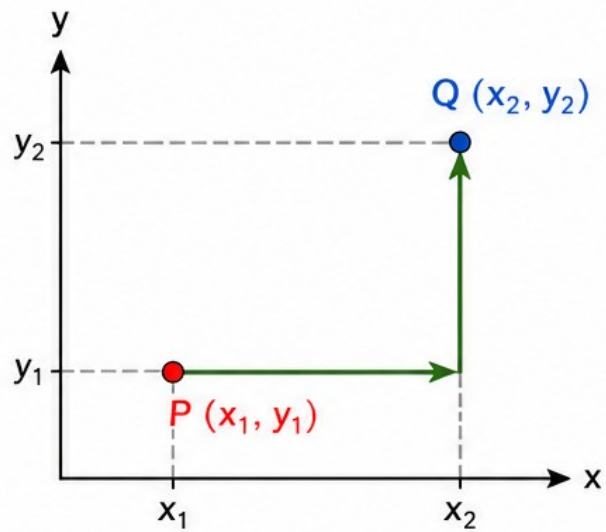


Straight line (as the crow flies) between P and Q.

$$d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

2. Manhattan Distance

$$d_M(P,Q) = |x_1 - x_2| + |y_1 - y_2|$$



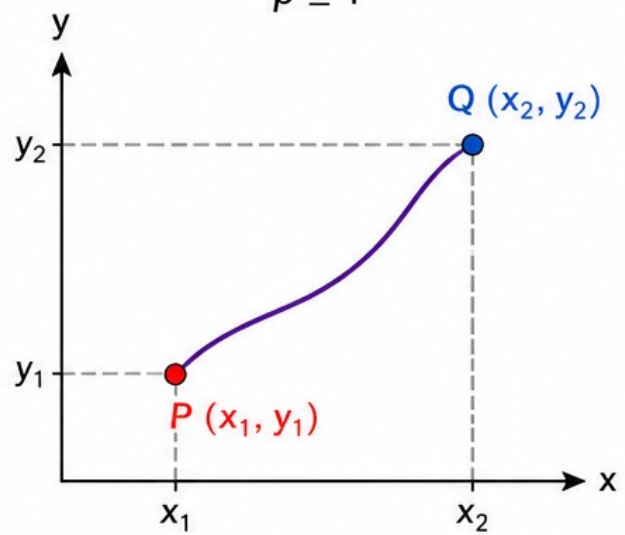
Path along the axes (like city blocks): horizontal + vertical.

$$d_M = |x_1 - x_2| + |y_1 - y_2|$$

3. Minkowski Distance

$$d_p(P,Q) = \left(|x_1 - x_2|^p + |y_1 - y_2|^p\right)^{\frac{1}{p}}$$

$$p \geq 1$$



Generalized distance parameterized by p. Special cases:

- $p = 1$ → Manhattan distance
- $p = 2$ → Euclidean distance
- $p \to \infty$ → Chebyshev (L_∞) distance

Note: $P\ (x_1, y_1)$ and $Q\ (x_2, y_2)$ are two points in 2D space.

KNN Regression:

- It is important to note that Linear Regression and KNN can both be used for regression, even though KNN is also commonly introduced as a classifier.
- For example, KNN Regression for house-price prediction, including hyperparameter tuning of K.

Problem

Consider the following dataset of houses. We use **Area** and **Number of Bedrooms** to predict the **House Price**. We want to predict the price of a new h1500 sq. ft. , $= (1500,\ \ 3)$ That is, **1500 sq.ft. and 3 bedrooms**.

House	**Area (sq.ft.)**	**No.of Bedrooms**	**Price (₹ lakh)**
1	800	2	40
2	1000	2	45
3	1200	3	52
4	1400	3	60
5	1600	3	68
6	1800	4	75
7	2000	4	82
8	2200	4	90

Original data

↓

Train/Test Split

↓

Feature Scaling using Standard Scaler

↓

Try K = 1, 2, 3, 4, 5

↓

Cross-validation

↓

Select K with minimum RMSE

↓

Train final KNN model

↓

Predict house price

Feature Scaling using StandardScaler

Why do we need feature scaling?

The two features have very different numerical ranges:

- Area: 800–2200
- Bedrooms: 2–4
- KNN is a **distance-based algorithm**. Therefore, Area could dominate the distance calculation.

➢ For example, without scaling, a **difference of 400 sq.ft. contributes much more than a difference of one bedroom.**

Therefore:

➢ Feature scaling is essential before applying KNN.

- We can use StandardScaler:

➢ One common method is Z-score standardization:

$$z=(x-\mu)/\sigma$$

where:

x= original value

μ= mean of the feature

σ= standard deviation of the feature

z= standardized value

After scaling, each feature will have approximately:

"Mean"=0

and

Standard deviation=1

Calculate mean of Area

Area values are:

$$800,1000,1200,1400,1600,1800,2000,2200$$

Mean:

$$\mu_{Area} = \frac{800 + 1000 + 1200 + 1400 + 1600 + 1800 + 2000 + 2200}{8}$$

μArea=1500

Calculate standard deviation of Area

For classroom simplicity, we can use the **population standard deviation**:

$$\sigma = \sqrt{\frac{\sum (x - \mu)^2}{N}}$$

The deviations are:

Sum:

$$\sum (x, -\mu)^2 = 1680000,$$

Therefore:

$$\sigma_{Area} = \sqrt{\frac{1{,}680{,}000}{8}}$$

$$= \sqrt{210{,}000}$$

$$\boxed{\sigma_{Area} \approx 458.26}$$

Area	$x - \mu$	$(x - \mu)^2$
800	-700	490000
1000	-500	250000
1200	-300	90000
1400	-100	10000
1600	100	10000
1800	300	90000
2000	500	250000
2200	700	490000

After scaling the feature Area:

House	Area	Scaled Area
1	800	-1.527
2	1000	-1.091
3	1200	-0.655
4	1400	-0.218
5	1600	0.218
6	1800	0.655
7	2000	1.091
8	2200	1.527

Notice that the Area values have now been transformed from: 800-2200 to approximately:−1.527 to 1.527

Scale Bedrooms

- Bedrooms: 2,2,3,3,3,4,4,4
- Mean:
- $\mu_{Bedroom} = \frac{2+2+3+3+3+4+4+4}{8}$
- $\boxed{\mu_{Bedroom} = 3.125}$
- Using population standard deviation:
- $\sigma_{Bedroom} \approx 0.7817$
- Therefore:
- $z_{Bedroom} = \frac{x-3.125}{0.7817}$
- For 2 bedrooms:
- $z = \frac{2-3.125}{0.7817} \approx -1.439$
- For 3 bedrooms:
- $z = \frac{3-3.125}{0.7817} \approx -0.160$
- For 4 bedrooms:
- z=0.78174−3.125≈1.120

Thus using : $z=(x-\mu)/\sigma$ the scaled bedroom will be:

House	Bedrooms	Scaled Bedrooms
1	2	-1.439
2	2	-1.439
3	3	-0.160
4	3	-0.160
5	3	-0.160
6	4	1.120
7	4	1.120
8	4	1.120

Now our final dataset becomes:

House	Area	Bedrooms	Scaled Area	Scaled Bedrooms	Price
H1	800	2	-1.527	-1.439	40
H2	1000	2	-1.091	-1.439	45
H3	1200	3	-0.655	-0.160	52
H4	1400	3	-0.218	-0.160	60
H5	1600	3	0.218	-0.160	68
H6	1800	4	0.655	1.120	75
H7	2000	4	1.091	1.120	82
H8	2200	4	1.527	1.120	90

Scale the New House

Our new house is:

Xnew=$(1500, 3)$

Area

$$z_{Area} = \frac{1500 - 1500}{458.26} = 0$$

Bedrooms

$$z_{Bedroom} = \frac{3 - 3.125}{0.7817} \approx -0.160$$

Therefore, the scaled new house is:

Xnew=(0,−0.160)

Now Calculate KNN Distance

- Suppose we want $K = 3$.
- Calculate the distance from the new house to every training house.

House	Price	Distance
H4	60	0.218
H5	68	0.218
H3	52	0.655

- For K=3:

Y^=60+68+52 /3

Y^=60 lakh

In the house-price KNN example, **cross-validation (CV)** is used to decide **which value of K gives the best generalization performance**.

What happens in our example?

- We have training data and we want to compare:

  K=1,2,3,4,5

  ➢ Instead of choosing one $K\ arbitrarily$, GridSearchCV tests each value using **3-fold cross-validation** because our code contains:

```
GridSearchCV(
    pipeline,
    param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error’
)
```

What does 3-fold CV mean?

- **3-fold cross-validation** means dividing the training data into 3 parts, using each part once for validation while training on the other two parts, calculating the RMSE for each round, and averaging the three RMSE values.

Suppose the training set has **6 houses**. We divide it into 3 groups:

Fold	Houses used for validation	Houses used for training
1	1, 2	3, 4, 5, 6
2	3, 4	1, 2, 5, 6
3	5, 6	1, 2, 3, 4

We train/evaluate KNN separately for each value:

```
K = 1 → evaluate model
K = 2 → evaluate model
K = 3 → evaluate model
K = 4 → evaluate model
K = 5 → evaluate model
```

Finally: Choose K with minimum CV RMSE

3-fold cross-validation means dividing the training data into 3 parts, using each part once for validation while training on the other two parts, calculating the RMSE for each round, and averaging the three RMSE values.

Try different values of K

We train/evaluate KNN separately for each value:

```
K = 1 → evaluate model
K = 2 → evaluate model
K = 3 → evaluate model
K = 4 → evaluate model
K = 5 → evaluate model
```

For each K, calculate the average RMSE.
Suppose we obtain:
Since **lower RMSE is better**:
K=3

K	Average CV RMSE
1	7.83
2	6.03
3	4.73
4	5.20
5	5.50

First: Calculate RMSE for each fold
Suppose for **K = 3**, 3-fold cross-validation gives these results:
Each RMSE is calculated separately using:

$$RMSE = \sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$$

So we have **three RMSE values**.

Fold	RMSE
Fold 1	5.05
Fold 2	4.62
Fold 3	4.57

Repeat for every K

Now we repeat the same process for:

$$K = 1,2,3,4,5$$

Suppose we get:

Therefore:

$$\boxed{K = 3}$$

has the **lowest CV RMSE**.

K	Fold 1	Fold 2	Fold 3	Average CV RMSE
1	7.2	8.1	7.8	7.70
2	5.9	6.2	6.0	6.03
3	5.05	4.62	4.57	4.75
4	5.1	5.3	5.2	5.20
5	5.4	5.6	5.5	5.50

The complete process in kNN

House Dataset

↓

Train-Test Split

↓

Scale Area + Bedrooms

↓

Choose possible K values (K = 1, 2, 3, 4, 5)

↓

Cross-validation

↓

Calculate RMSE for each K

↓

Choose K with lowest RMSE

↓

Best K = 3

↓

Train final KNN model

↓

Comparison of kNN and linear regression

- Linear Regression and KNN can both be used for regression, even though KNN is also commonly introduced as a classifier.
- Key difference: Linear Regression learns a global equation, whereas KNN learns locally from nearby observations.
- For example, if the **relationship between area and price is** approximately a **straight line, Linear Regression may work very well**.
- If the relationship is highly **irregular or nonlinear, KNN Regression** can capture local patterns without explicitly fitting a particular equation.

Aspect	Linear Regression	KNN Regression
Type	Parametric	Non-parametric
Learning approach	Model-based	Instance-based
Model	Fits a mathematical line/plane	Uses nearby training observations
Assumption	Assumes approximately linear relationship	Makes no specific functional-form assumption
Training	Estimates coefficients	Essentially stores training data
Prediction	Uses learned equation	Averages the K nearest target values
Effect of K	Not applicable	Controls smoothness
Sensitive to distance	No	Yes
Feature scaling	Usually less critical	Very important
Outliers	Can strongly affect fitted line	Can affect local predictions
Interpretability	High	Relatively low
Prediction speed	Fast	Can be slower for large datasets
Best suited for	Linear relationships	Complex/local relationships

Data normalization/standardization in KNN

- What is data normalization?
- Why is it required?

Performance of KNN is strictly affected by 3 factors:

- No. of nearest neighbors (value of k)
- Distance metric
- Decision rule

- What happens if k is very small?
- What happens if k is very big?

Choice of Distance metric

- Depends on the type of independent metric in the data set

➢ What happens if the points are at different distances?

- It is important that how you choose the distance metrics. It should be independent of the value of the training dataset.

➢ The training dataset with bigger number doesn’t mean that it has to be given importance

Decision rule

- KNN works better for a dataset that is in low dimensions why?

Weighted KNN

- What is weighted knn?
- What is the principle behind weighted knn?

➢ k closest neighbors are assigned to the test instance with higher weights in the decision as compared to neighbors that are farther away from test instance

- How do we choose the value of the weights?
- What is we do not assign weights?

➢It works like normal KNN. All are (each neighborhood) weighted equally

➢If you are giving weights then give more weights to the points which are nearer

(closer neighbor will have greater influence than neighbors which are farther away

Algorithm of Weighted KNN

- **Input:** Training dataset T, Distance metric d, Weighted function w, test instance t, nearest neighbor k
- **Output:** Predicting class or category
- Steps: First 2 steps are same as normal KNN
- We have to apply weight for k selected nearest instance
- How do we compute the weight?

Inputs: Training dataset 'T', Distance metric 'd', Weighting function $w(i)$, Test instance 't', the number of nearest neighbors 'k'

Output: Predicted class or category

Prediction: For test instance t.

1. For each instance 'i' in Training dataset T, compute the distance between the test instance t and every other instance 'i' using a distance metric (Euclidean distance).
   [Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as dist $((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
   [Categorical attributes (Binary) - Hamming Distance: If the values of two instances are the same, the distance d will be equal to 0. Otherwise $d = 1$.]
2. Sort the distances in the ascending order and select the first 'k' nearest training data instances to the test instance.
3. Predict the class of the test instance by weighted voting technique (Weighting function $w(i)$) for the k selected nearest instances:
   - Compute the inverse of each distance of the 'k' selected nearest instances.
   - Find the sum of the inverses.
   - Compute the weight by dividing each inverse distance by the sum. (Each weight is a vote for its associated class).

Inputs: Training dataset 'T', Distance metric 'd', Weighting function $w(i)$, Test instance 't', the number of nearest neighbors 'k'

Output: Predicted class or category

Prediction: For test instance t.

1. For each instance 'i' in Training dataset T, compute the distance between the test instance t and every other instance 'i' using a distance metric (Euclidean distance).
   [Continuous attributes - Euclidean distance between two points in the plane with coordinates (x_1, y_1) and (x_2, y_2) is given as dist $((x_1, y_1), (x_2, y_2)) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]
   [Categorical attributes (Binary) - Hamming Distance: If the values of two instances are the same, the distance d will be equal to 0. Otherwise $d = 1$.]
2. Sort the distances in the ascending order and select the first 'k' nearest training data instances to the test instance.
3. Predict the class of the test instance by weighted voting technique (weighting function $w(i)$) for the k selected nearest instances:
   - Compute the inverse of each distance of the 'k' selected nearest instances.
   - Find the sum of the inverses.
   - Compute the weight by dividing each inverse distance by the sum. (Each weight is a vote for its associated class).
   - Add the we[illegible]
   - Predict the class by choosing the class with the maximum vote.

8:56

and then we find the sum of the inverses.